

MLIR-MRacle: A Metamorphic Fuzzer for Detecting Concurrency Bugs in OpenMP MLIR Lowering

Aaron Gabayoyo

1 Introduction

Multi-Level Intermediate Representation (MLIR) is an extensible compiler framework whose dialects, logical groupings of operations, types, and attributes, support multiple levels of abstraction and has become widely adopted in the industry. Dialects such as `async` and `OpenMP` introduce concurrency to MLIR, enabling the compilation of concurrent programs for high-performance computing, machine learning, and safety-critical real-time systems.

Concurrency in programs rely on tasking and thread scheduling, which introduce non-determinism through instruction interleaving. The possible interleavings are defined by the target hardware memory model, so behaviour is not uniform across targets.

Because MLIR is increasingly used in safety-critical systems, testing and verification must specifically target concurrency. Concurrency bugs, such as data races, atomicity violations, and ordering violations, can remain hidden at the dialect level because MLIR’s abstractions hide target-specific memory-model details away from the developer. Thus, testing concurrency dialects and their transformations at the MLIR stage is essential before they propagate into invalid concurrent behaviour.

However, applying the current MLIR testing approaches to concurrency programs and dialects raises two challenges:

1. **Magnification of the Oracle Problem.** An oracle is the mechanism that judges whether an observed output is correct. Used commonly in testing, deriving this output in deterministic programs is already a known problem in the testing space. For non-deterministic programs, the oracle must now enumerate the entire set of all possible outcomes, making oracle construction substantially harder, more error-prone and impractical.
2. **Single-threaded determinism assumptions.** Many testing techniques assume sequential execution with a fixed global order of operations. This assumption breaks down for concurrent programs, where thread scheduling introduces nondeterministic interleavings and hardware memory models permit varying re-orderings per hardware. As a result, such techniques can misinterpret valid executions as failures, miss interleaving-specific defects, or alter the very behaviour they intend to preserve.

To address these issues, this report presents MLIR-MRacle, a fuzzing tool for MLIR’s OpenMP dialect. MLIR-MRacle applies semantics-preserving transformations to concurrent programs using metamorphic relations (MRs), sidestepping the oracle problem for concurrency entirely. Importantly, these transformations are also defined to be concurrency-safe and memory-model-safe, ensuring that generated programs are well-defined under the relevant memory model and thus support reliable comparison and reasoning about their behaviour, unlike other existing tools.

2 Background and Existing Work

2.1 Traditional MLIR Testing

Overview: Traditional in-repo dialect testing relies on two complementary approaches with handwritten programs. The first uses `mlir-opt` and the `lit` framework, passing handwritten MLIR programs through the transformations under test and comparing the resulting string output using `FileCheck`, an LLVM/MLIR testing tool for matching specified patterns and expected outputs. The second uses `mlir-cpu-runner` to execute handwritten MLIR programs to compare the actual and expected outcomes, again through `FileCheck`.

Limitations and Motivation: Although useful for simple test cases and sanity checks, these approaches are severely limited: manually generating programs and reasoning about expected outputs is hard, and `FileCheck` only accommodates checks for a single, deterministic output. These limitations directly motivate MLIR-MRacle, which, like other MLIR testing tools, aims to improve upon the standard testing infrastructure. A metamorphic fuzzer was chosen precisely because it replaces fixed output matching with relational checks across generated programs, addressing the inability to handle nondeterministic outputs and manually specified expected results.

2.2 Fuzzing in MLIR

Overview: Fuzzing in MLIR automatically generates or mutates the intermediate representation (IR) in MLIR programs to find unintended compiler crashes and hangs. Representative tools include MLIR-Smith, a grammar-based generator; Forge, a mutation- and coverage-guided fuzzer; and Ratte, which leverages dialect specifications to generate semantically valid IR. Unlike `FileCheck` tests, fuzzers only require that the compiler does not crash or hang, enabling large-scale testing with minimal manual effort.

Limitations and Motivation: However, current MLIR fuzzers have two major limitations for concurrent programs: they assume their transformations preserve validity under concurrency and ignore memory models entirely. As a result, when fuzzing concurrent IR, most generated programs are semantically invalid or have undefined behaviour (UB), removing MLIR fuzzers’ ability to attribute crashes or semantic changes to its transformations alone. MLIR-MRacle addresses this by making the set of ap-

plied transformations concurrency-aware and memory-model-dependent, ensuring that generated concurrent programs are well-defined under the target memory model.

2.3 Concurrency Testing

Overview: Concurrency testing seeks to expose bugs caused by thread interleavings, synchronization, and memory-ordering effects. Existing approaches range from systematic techniques, such as model checking, which explore possible executions under a defined concurrency and memory model, to execution-based techniques that repeatedly run concurrent programs and observe their behaviours. Representative tools include CHES and CDSChecker for systematic exploration, and C4 for compiler-focused testing through repeated execution and outcome-set comparison.

Limitations and Motivation: Despite these established approaches, there is currently no concurrency testing tool designed specifically for MLIR concurrency constructs. Model checkers target particular language-level concurrency primitives and memory models - none of which are compatible with MLIR concurrency constructs considered in this work. Execution-based approaches such as C4 provide a useful precedent for compiler testing, but are likewise designed around a particular language and memory model. As MLIR is multi-level and multi-target ecosystem, this requires a testing framework that can accommodate different memory models and target-specific concurrency semantics. This gap motivates a concurrency testing framework compatible with MLIR and its target-specific concurrency semantics.

2.4 Metamorphic Testing

Overview: Metamorphic testing addresses the difficulty of defining an exact expected output by testing relationships between multiple executions. A test is transformed in a way that should preserve or predictably alter its behaviour, and the resulting executions are checked against the expected relation. Approaches such as GraphicsFuzz and C4 demonstrate the use of metamorphic testing in automated compiler testing, with C4 combining metamorphic relations with fuzzing and outcome-set comparison for concurrent programs.

Limitations and Motivation: The effectiveness of metamorphic testing depends on generating sufficiently diverse related tests and defining relations that capture meaningful compiler behaviours. C4 demonstrates how fuzzing can address the former by automatically producing large numbers of metamorphically related programs, while outcome-set comparison avoids requiring a single expected result. However, existing approaches are developed around specific languages and semantic models rather than a general intermediate representation such as MLIR which lowers to many different hardware targets. This motivates a metamorphic fuzzing approach that can support testing under different memory models and concurrency semantics.

3 MLIR-MRacle

MLIR-MRacle is a metamorphic fuzzing framework for testing compiler handling of concurrent programs. The framework uses the OpenMP dialect as a vehicle for exploring concurrency within MLIR, providing a concrete setting in which metamorphic transformations can be applied to concurrent programs. The tool is designed around the generation of related test cases, their compilation and execution, and the comparison of their resulting behaviour to identify potential correctness issues.

3.1 System Overview

Figure 1 details the overall architecture and key components of the MLIR-MRacle metamorphic fuzzing tool. At a high level, the pipeline starts from a source MLIR file — containing litmus-style OpenMP constructs — drawn from the corpus. This source file is metamorphically transformed, producing a related but distinct transformed MLIR file. Each of these two files is then cloned into five copies and independently agitated, to increase the diversity of execution outcomes explored, before being executed repeatedly. Finally, the resulting outcome sets are collected and compared by the oracle.

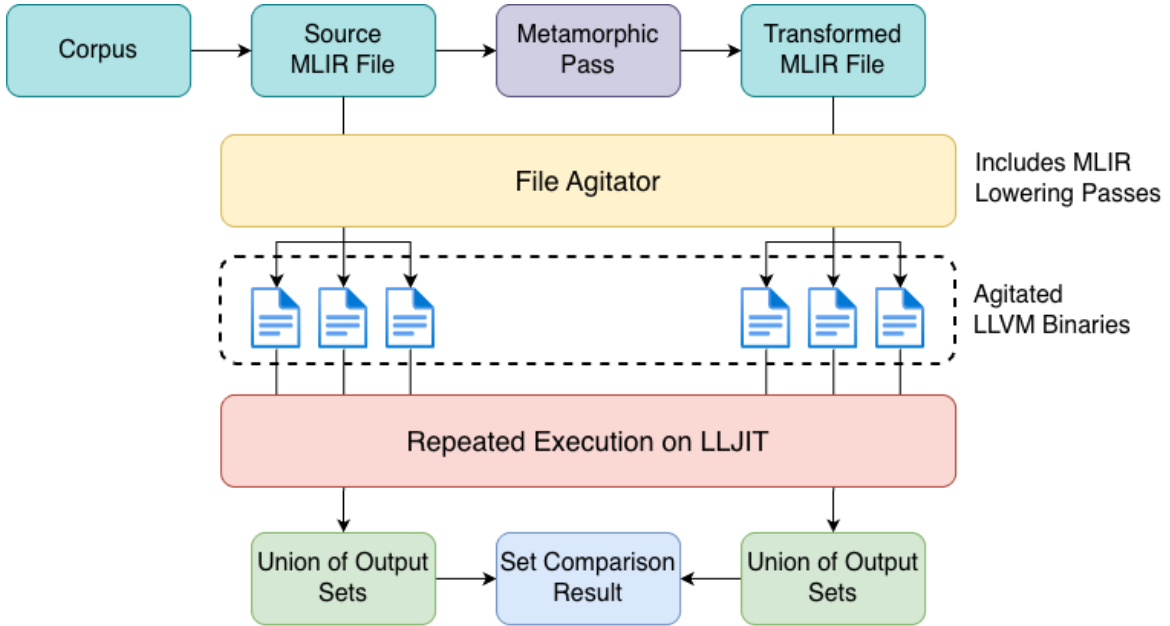


Figure 1: Architecture diagram of MLIR-MRacle

3.2 The Corpus

To start, MLIR-MRacle has a collection of source programs, referred to as the corpus, which provides the starting point for its metamorphic testing process. The corpus consists of a set of seeds, where each seed is an individual input program that the metamorphic fuzzer can randomly select, mutate, and transform to generate test cases. In

this work, the seeds are concurrent MLIR programs in the OpenMP dialect (as shown in 1), derived and converted directly from base litmus tests, which are small, deliberately constructed concurrent programs used to study and distinguish the behaviours permitted by a memory model.

Litmus tests are particularly well suited as a source of seeds because they are deliberately designed to exercise interesting and subtle concurrency behaviours, while their widespread use provides an established collection of tests, avoiding the need to construct OpenMP concurrency programs from scratch.

The litmus tests were obtained from the C4 corpora, the Linux repo’s memory-model and concurrency tests, and SV-COMP, and were converted into OpenMP-dialect MLIR semi-manually with the assistance of AI agents. During this conversion, we remove race conditions, since the original litmus tests may deliberately rely on undefined behaviour (UB) to explore weak-memory behaviours, as fuzzers normally presume their programs are well-defined. We also deliberately constrain program return values towards post-join, deterministic results wherever possible, reducing dependence on scheduler timing and avoiding introducing unnecessary scheduler noise for comparison’s sake.

```

1 module {
2   func.func @main() -> i64 {
3     %c0 = arith.constant 0 : index
4     %c0_i64 = arith.constant 0 : i64
5     %c1_i64 = arith.constant 1 : i64
6
7     %x = memref.alloc() : memref<1xi64>
8     memref.store %c0_i64, %x[%c0] : memref<1xi64>
9
10    omp.parallel {
11      omp.sections {
12        omp.section {
13          // P0: atomic_store(x, 1); r0 = atomic_load(x);
14          omp.atomic.write %x = %c1_i64 memory_order(relaxed) : memref<1xi64>, i64
15          %r0_mem = memref.alloca() : memref<1xi64>
16          memref.store %c0_i64, %r0_mem[%c0] : memref<1xi64>
17          omp.atomic.read %r0_mem = %x memory_order(relaxed) : memref<1xi64>, memref
18             ↪ <1xi64>, i64
19          %r0 = memref.load %r0_mem[%c0] : memref<1xi64>
20          omp.terminator
21        }
22        omp.terminator
23      }
24      omp.terminator
25    }
26
27    %x_val = memref.load %x[%c0] : memref<1xi64>
28    memref.dealloc %x : memref<1xi64>
29    return %x_val : i64
30  }

```

Listing 1: Example of a Litmus-style MLIR OpenMP Program

3.3 The Metamorphic Pass

The tool defines a dedicated metamorphic pass to apply transformations during metamorphic testing, leveraging MLIR’s pass infrastructure to make these transformations easier to apply within the existing compilation pipeline. The transformations are implemented as MLIR rewrites, allowing the pass to modify programs directly within the existing IR infrastructure.

The transformations extend a common transformation class, with each implementation carrying the information required by the metamorphic pass to determine when it can be applied. Looking at the ARMv8-specific transformations shown in Listing 2, each is defined by a name, the memory model under which it is permitted, the rewrite function used to apply it, and the relation it represents. The relation may be equality, subset, or superset, and is used by the pass alongside the memory model to determine which transformations are applicable and which combinations should be excluded. Keeping these decisions as part of each transformation allows the pass to remain independent of individual transformation implementations, while making it straightforward to extend the transformation set to new memory models or architectures.

```
1 const FunctionTransform kArmTransforms[] = {  
2     FunctionTransform("commute-relaxed-read-write", "armv8",  
3                       &tryCommuteRelaxedReadWrite,  
4                       mlir_mracle::OutcomeRelation::Equality),  
5     FunctionTransform("commute-relaxed-write-write", "armv8",  
6                       &tryCommuteRelaxedWriteWrite,  
7                       mlir_mracle::OutcomeRelation::Equality),  
8 };
```

Listing 2: Definition of ARMv8 Transformations

For every input program, the pass filters the available transformations by the current memory model and their relation constraints. It then randomly selects one of the remaining transformations and attempts to apply its rewrite to a randomly selected program from the corpus. If the selected rewrite cannot be applied, another applicable transformation is selected and attempted. This continues until a transformation succeeds, resulting in a transformed MLIR file, or no applicable transformations remain.

3.4 Agitation

Once two related programs are produced by the metamorphic transformation, MLIR-MRacle applies agitation, namely the deliberate introduction of controlled variation in program representations, compilation decisions, and execution conditions, throughout the pipeline to expose as many distinct execution states as possible before set comparison. This is particularly important because the programs may exhibit nondeterministic execution with multiple possible valid outcomes. Consequently, observing no difference in a single run is insufficient evidence of equivalence; agitation is therefore used to broaden the set of states considered.

The agitation process is illustrated in Figure 2. It begins by cloning each program at the MLIR level, creating independent compilation and execution instances. This allows each clone to follow a different perturbation path while retaining a common basis for comparison. Lightweight jitter is then introduced by inserting random `scf.while` operations, varying program timing and execution paths while preserving structural similarity. The clones are subsequently lowered to LLVM, where transformations such as basic-block shuffling add further diversity in control flow and code layout. Finally, the variants are compiled with varying JIT optimisation levels, OpenMP configurations, and thread counts, further widening the range of execution conditions represented in the collected sets.

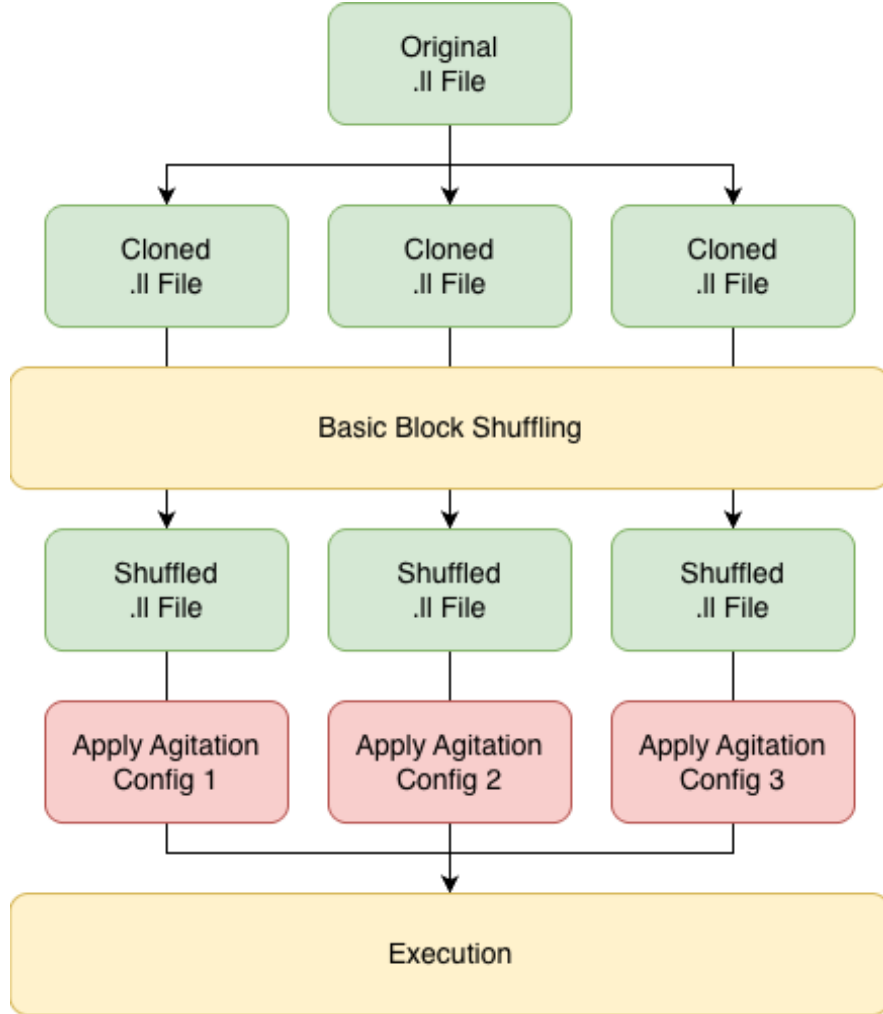


Figure 2: Agitation Process

3.5 Execution and the Oracle

3.5.1 Execution

After lowering the source and transformed programs into independently executable clones, MLIR-MRacle executes them through MLIR’s LLJIT engine, which provides a

direct and lightweight execution path without requiring a separate runtime infrastructure. Each clone is run for a number of trials and resulting outcomes are aggregated into a singular, final outcome set, as shown in Figure 3. Unlike many regular fuzzers, multiple executions are required here due to the nondeterminism in the programs. In parallel, MLIR-MRacle performs a single-threaded determinism check on the source program as an early-exit case, ensuring that obvious bugs and issues are caught at a low cost.

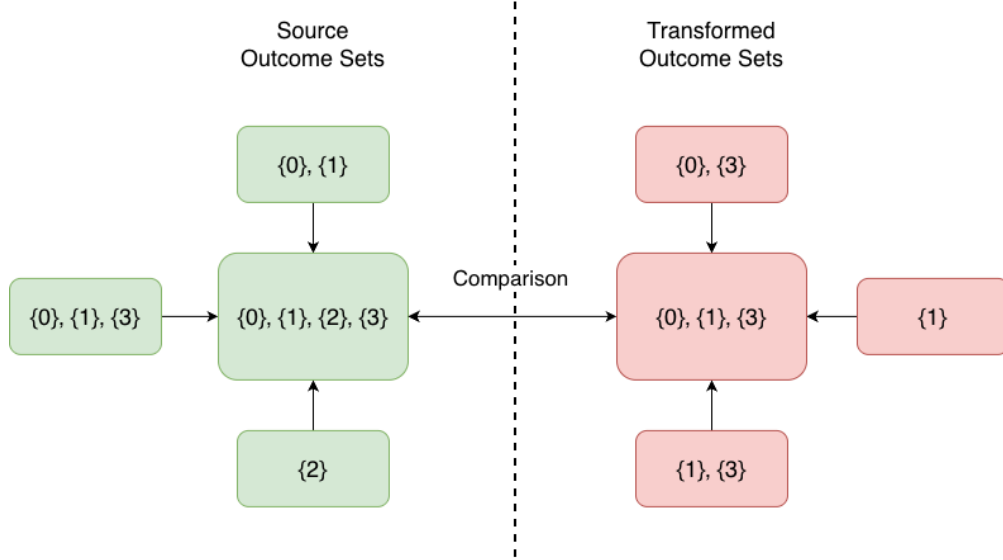


Figure 3: Total Outcome Set Comparison

3.5.2 Oracle

Now that the outcome sets are retrieved, the oracle first handles the simple cases using relation-specific set comparison, depending on the type of metamorphic relation (equality, subset or superset). Satisfaction of the relation yields an OK result while the opposite yields a failure; this would be cases where a commonly appearing outcome set is novel to the source, or is missing from the transformed.

To account for rare outcome sets, MLIR-MRacle applies a Poisson-based bound to distinguish meaningful observations from events that are too infrequent to support a reliable conclusion. Such cases yield a WARN result rather than an immediate verdict, reflecting the fact that finite executions cannot establish whether a rare outcome is merely a nondeterministic event or evidence of a violation. Additionally, any value that appears and is not found at all in the source set is immediately a WARN: a genuinely new value is substantially riskier than a novel outcome set, since a new outcome may consist entirely of values already observed in other outcomes and differ only as a unique combination.

A WARN triggers a rerun to make the result more conclusive: fresh source and transformed clones are generated and re-agitated to obtain additional independent executions, after which the accumulated observations are re-evaluated using the same oracle. Reruns continue only while the result remains inconclusive, up to a maximum rerun

budget. A persistent violation becomes FAIL, while evidence satisfying the relation becomes OK; if the budget is exhausted without a conclusive classification, the test returns a WARN result.

For reproducibility and downstream analysis, MLIR-MRacle records the execution outcomes, oracle classification, and relevant configuration information in JSON files on disk.

4 Findings

After running multiple campaigns, MLIR-MRacle was able to identify a verification issue in the OpenMP dialect, where the verifier failed to match spec by not validating nested critical operations. This resulted in the compilation of even simple programs that would hang indefinitely; the issue was subsequently reported upstream and the corresponding pull request was merged. The testing also reached a currently known limitation involving OpenMP operations that do not operate on memref, which has been deliberately postponed but demonstrates that the fuzzer is reaching bug-adjacent parts of the dialect. Finally, several behaviours warrant further investigation, particularly concerning the relaxation and restriction of memory ordering attributes on operations: despite imposing opposing constraints, applying either tends to reduce the number of observed outcomes. These observations are not yet understood as defects, but might be interesting to go into further detail.

5 Limitations & Future Work

MLIR-MRacle currently has several limitations that constrain the breadth and completeness of its testing. First, its current scope is limited to the OpenMP dialect, so the approach does not yet apply to other MLIR concurrency dialects or programming models. The input corpus is also hard-coded, without coverage-guided or other feedback-driven fuzzing to steer generation towards unexplored program regions.

Furthermore, because executions are nondeterministic, repeated sampling cannot guarantee that all reachable behaviours will be observed. Some outcomes may not occur within the allotted executions, while others may occur only rarely and require repeated reruns before they can be classified. Consequently, the execution-based oracle provides no completeness guarantees, and some tests may remain inconclusive even after multiple executions.

Finally, although the current metamorphic transformations provide broad coverage, more targeted and aggressive transformations, particularly those exploiting architecture-specific features such as ARM memory-ordering behaviour, could expose additional compiler and concurrency bugs.

These limitations suggest several directions for future work. Extending the framework beyond OpenMP to other MLIR concurrency dialects would greatly broaden its applica-

bility. Coverage-guided corpus generation could improve exploration beyond the current fixed set of programs, while richer and architecture-specific metamorphic transformations could target more subtle behaviours. To complement the inherent incompleteness of execution-based testing, integration with a model checker or another systematic exploration technique (e.g. deterministic scheduling, if feasible) could provide stronger guarantees about MLIR-MRacle’s results. Emulation would also allow MLIR-MRacle to evaluate programs for architectures other than the host device, enabling testing of target-specific concurrency semantics and transformations without requiring dedicated hardware.

6 Conclusion

MLIR-MRacle shows that metamorphic testing is a practical way to probe compiler transformations in concurrent settings, where nondeterminism makes conventional testing especially hard. It targets a largely unexamined gap in MLIR: whether memory-model and concurrency semantics are preserved across passes and lowering stages. By mutating OpenMP programs and comparing their behaviour, the framework offers a means of exercising concurrency-related compiler behaviour and highlighting violations of expected relations. This approach was able to identify a genuine bug in MLIR, indicating that even a small-scale application of the technique can find real issues. This represents an initial step toward more systematic testing of concurrency-preserving transformations in MLIR, rather than a comprehensive evaluation.